

Hero Cycles Pricing Engine

Proper UI Design & Implementation Specification

1. Product UI Vision

Design the application as a specialized **Sales Pricing Workspace**, not as a conventional inventory or admin dashboard. The primary purpose is to help Hero Cycles sales users manage parts, understand price history, build or customize cycle configurations, and immediately understand the price impact of part changes.

Core product experience:

Parts → Price Changes → Pricing Engine → Configuration Impact → Live Cycle Price

The UI should feel like a premium product configurator combined with a pricing intelligence tool.

2. Primary User

The primary user is a Hero Cycles salesperson who needs a faster alternative to Excel for working with cycle parts, prices and configurations. The interface should prioritize speed, clarity and confidence when preparing or modifying a cycle configuration.

3. Design Principles

- **Sales-first:** optimize for configuring and pricing cycles, not database administration.
- **Visual clarity:** make component prices, quantities and total price immediately understandable.
- **Live feedback:** price changes should be visible as soon as the user changes components or quantities.
- **Price transparency:** show how the final total was produced.
- **Impact awareness:** show which configurations are affected when a part price changes.
- **Low friction:** minimize unnecessary forms and navigation.
- **Distinctive:** avoid generic inventory-dashboard patterns.

4. Visual Identity

Use a premium, modern palette based on **deep teal, teal, mustard/gold, corall/peach and soft purple**. The primary canvas should remain light and spacious.

Deep Teal #0F3D3E – brand/navigation/strong sections
Teal #148A86 – active/positive states
Mustard #F59E0B – primary actions/key totals
Coral #FB7185 – warnings/price increases/attention
Purple #8B5CF6 – secondary information
Canvas #F8FAFC – application background
Surface #FFFDF8 – cards and content surfaces
Text #1F2937 – primary text

Color must communicate meaning. Do not make every component colorful. Keep the interface predominantly light, with deep teal providing brand contrast and accent colors used selectively.

5. Application Shell

Use a persistent left navigation on desktop and a compact navigation pattern on smaller screens.

[illegible]

[illegible]

Navigation order should emphasize Cycle Builder and the pricing workflow rather than generic administrative functions.

6. Navigation

Dashboard — pricing overview and recent activity

Cycle Builder — primary workflow for building/customizing cycles

Configurations — predefined and custom configurations

Parts — reusable parts library

Price History — historical price timeline

Price Impact — affected configurations after price changes

7. Dashboard

The dashboard should be a **Pricing Pulse**, not an analytics-heavy corporate dashboard.

```
■■■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■  
■ Total Parts ■ ■ Configurations ■ ■ Price Changes ■ ■ Affected ■  
■ 128 ■ ■ 42 ■ ■ 15 ■ ■ Confs: 18 ■  
■■■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■
```

Below the summary: Pricing Pulse, recent price changes, top price increases, and recent configurations.

8. Cycle Builder — Primary Screen

The Cycle Builder is the centerpiece of the application. It should feel like a product configurator rather than a form.

Recommended layout:

Left/Main: configuration identity + selected components + quantity controls.

Right: live Price Breakdown + calculation date.

Bottom: sticky total and Save Configuration action.

BUILD YOUR CYCLE

Configuration: Mountain Pro Custom

```

Frame Mountain Frame Qty 1
Gear Set 18-Speed Gear Set Qty 1
Tyre Mountain Tyre Qty 2
Brake Hydraulic Brake Qty 2
[ + Add Component ]

```

PRICE BREAKDOWN ■24,800
[Save Configuration]

9. Configuration Types

Predefined: standard reusable cycle configuration created by authorized users.

Custom: user-created configuration, either from scratch or derived from a predefined configuration.

Use visually distinct but restrained badges such as PREDEFINED and CUSTOM. Keep the original predefined configuration immutable when the salesperson chooses Customize.

10. Customize Interaction

The primary interaction should be one clear action: **Customize.**

PREDEFINED CONFIGURATION
↓
[Customize]

↓

↓

S

11. Component Selector

Use a searchable, visually clean selector rather than a large unfiltered dropdown.

Search parts: [mountain tyre_____]

Mountain Tyre ■1,350 ACTIVE [Add]

Road Tyre ■1,450 ACTIVE [Add]

Old Tyre ■900 INACTIVE [Disabled]

Show the currently applicable price for the calculation date. Inactive parts may be visible for context but cannot be added to a new configuration.

12. Quantity Control

Use a compact quantity stepper with clear minus/plus actions and direct numeric entry where appropriate.

Tyre [-] 2 [+] ■1,350 × 2 = ■2,700

13. Live Price Breakdown

The breakdown should always remain visible while configuring a cycle.

[illegible]

When a quantity changes, the UI should immediately update the component total and grand total.

14. Sticky Pricing Summary

On the Cycle Builder, keep a sticky summary area so the salesperson does not need to scroll to see the current price.

4 Components • 6 Parts • PRICE ■11,800 [Save Configuration]

15. Parts Library

Use a hybrid card/table approach. Cards make the product feel distinctive and help users discover parts; a compact table can be provided for dense management.

PART CARD
Mountain Tyre
TYRE
■1,350
↑ ■150 recently
Used in 8 configurations
[View Details]

Include search, category filter, status filter and Add Part.

16. Part Details

Show the current part information first, then make price history the main supporting information.

Mountain Tyre
Category: Tyre Status: Active
Current Price: █1,350

[+ Add New Price]

17. Price History

Use a timeline to communicate price evolution visually while retaining exact values.

█950 ●██████████ 01-Jan-2025
█1,050 ●██████████ 01-Jul-2025
█1,200 ●██████████ 01-Jan-2026
█1,350 ●██████████ 01-Jun-2026 → PRESENT

Also show previous-price change, percentage change, effective dates and the ability to add a future price.

18. Price Change Interaction

After saving a price change, show a confirmation that clearly communicates old price → new price and provides a direct path to the affected configurations.

PRICE UPDATED

Mountain Tyre
█1,200 → █1,350
+█150

8 configurations affected
[View Price Impact]

19. Price Impact — Signature Feature

Price Impact is the most distinctive business feature. It should make the effect of a part-price change immediately understandable.

PRICE IMPACT

Mountain Tyre █1,200 → █1,350 +█150

Mountain Pro █24,500 → █24,800 +█300
Trail Master █28,200 → █28,500 +█300
City Rider █18,400 → █18,550 +█150

Use coral/peach for increases, teal for positive/neutral states and restrained visual emphasis for large impacts.

20. Historical Price Calculation

Configuration details should provide a 'Price as of' control. The backend determines the applicable historical prices.

PRICE AS OF: [15-Aug-2026 ▼]

Frame █5,000
Tyre █220 × 2
Brake █800 × 2
TOTAL █...

21. States and Feedback

- **Loading:** use skeletons that preserve page structure.
- **Empty:** explain the state and provide one obvious next action.
- **Success:** use concise confirmation/toast plus relevant next action.
- **Error:** identify the problem and how the user can resolve it.
- **Missing price:** identify the exact part and calculation date.
- **Overlap:** identify the conflicting price period.

- **Inactive:** show disabled state clearly and prevent selection.

22. Responsive Behavior

Desktop is the primary target because the workflow is sales/business oriented. On smaller screens, stack the Cycle Builder sections vertically, move the Price Breakdown below the components, and keep the total/save action accessible.

23. Component Architecture

```
components/  
■■■ layout/Sidebar.tsx  
■■■ layout/Header.tsx  
■■■ dashboard/PricingPulse.tsx  
■■■ dashboard/RecentPriceChanges.tsx  
■■■ builder/CycleBuilder.tsx  
■■■ builder/ComponentSelector.tsx  
■■■ builder/QuantityStepper.tsx  
■■■ builder/PriceBreakdown.tsx  
■■■ configurations/ConfigurationCard.tsx  
■■■ configurations/ConfigurationList.tsx  
■■■ parts/PartCard.tsx  
■■■ parts/PartTable.tsx  
■■■ parts/PriceHistoryTimeline.tsx  
■■■ impact/PriceImpactSummary.tsx  
■■■ impact/PriceImpactTable.tsx  
■■■ ui/...
```

24. Route Structure

```
/dashboard  
/cycle-builder  
/configurations  
/configurations/[id]  
/configurations/new  
/parts  
/parts/[id]  
/price-history  
/price-impact
```

25. UI-to-API Responsibilities

UI action	Backend responsibility
Load parts	Return active/inactive parts and applicable prices
Add price	Validate period and persist new PartPrice
Calculate cycle	Return component breakdown and total
Change quantity	Recalculate component and total
Customize	Create separate CUSTOM configuration
View history	Return effective-dated price records
View impact	Calculate affected configurations and price differences

26. Accessibility and Usability

Maintain readable contrast, visible focus states, meaningful labels, keyboard-accessible controls and clear error text. Do not rely on color alone to communicate status; pair color with text, icons or badges.

27. MVP UI Scope

The UI is complete when the salesperson can browse predefined configurations, customize one without modifying the original, add/remove parts, change quantities, see a live component-wise price, save a custom configuration, manage part prices, view price history, and inspect the price impact of a part-price change.

28. Final Design Rule

The application should immediately communicate: **'I can build a Hero Cycles configuration here and understand exactly how its price changes.'** Every major UI decision should support that message. Avoid unnecessary dashboard widgets, generic inventory patterns and decorative UI that does not improve the sales workflow.